Lezione 2

Regressione

Paolo Bosetti

Indice

1 Regressione lineare **1**
1.1 Modello lineare univariato 1
1.2 Estrapolazione 13
1.3 Regressione ai minimi quadrati 17
1.4 Regressione lineare generalizzata 22

1 Regressione lineare

1.1 Modello lineare univariato

Supponiamo di avere una relazione tra due variabili x e y del tipo:

$$y = a + bx + cx^2$$

Realizziamo un esperimento che ci consenta di scegliere i valori di x e misurare i corrispondenti valori di y.

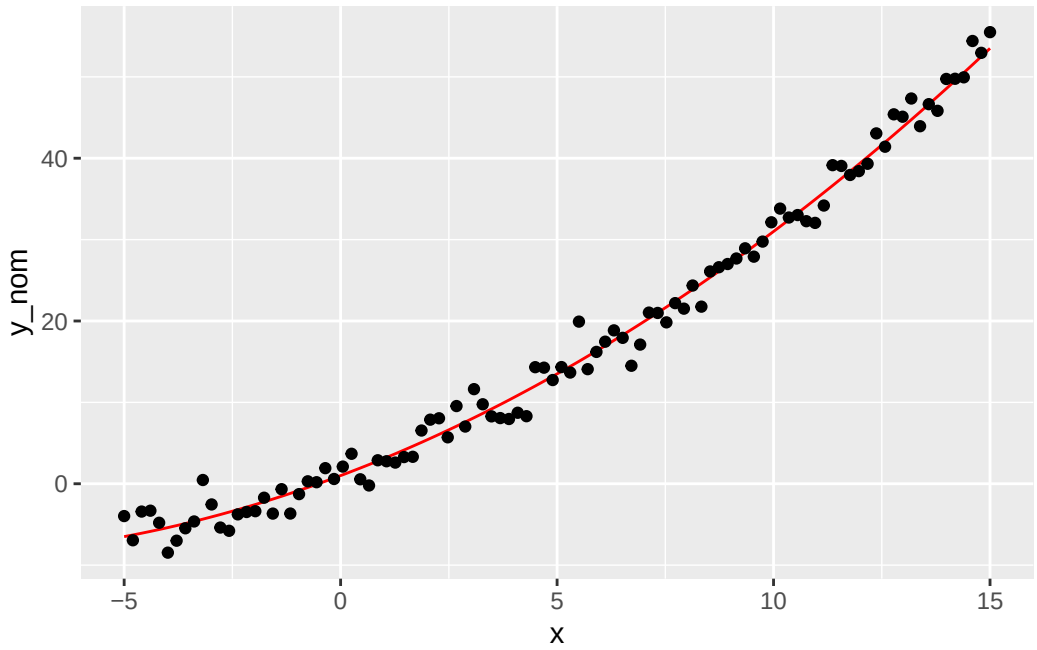
Cominciamo con il simulare l'esperimento, generando dei dati sintetici. In particolare, generiamo 100 valori di x equispaziati tra -10 e 10, e calcoliamo i corrispondenti valori di y secondo la relazione sopra (`y_nom`, valore *nominale*), aggiungendo un po' di rumore casuale normale con media nulla e deviazione standard 2 (`y`, valore *misurato*).

```
set.seed(0)
N <- 100
a <- 1
b <- 2
c <- 0.1

df <- tibble(
  x = seq(-5, 15, length.out=N),
```

```r
  y_nom = a + b*x + c*x^2,
  y = y_nom + rnorm(N, 0, 2)
)

# in rosso la relazione nominale, in nero le "misure"
df %>% ggplot(aes(x=x)) +
  geom_line(aes(y=y_nom), color="red") +
  geom_point(aes(y=y))
```



Il nostro obiettivo è stimare i parametri a, b e c a partire dai dati osservati.

Il primo passo è definire un **modello statistico** che corrisponda alla relazione $y = f(x)$ ma includa anche il disturbo casuale:

$$y_i = a + bx_i + cx_i^2 + \epsilon_i, \quad \epsilon_i \sim N(0, \sigma^2)$$

Si noti che il modello si dice **lineare** perché è lineare nei **parametri** a, b e c, anche se non è lineare nella variabile x.

In R, il primo passo è trasformare il modello in una formula. La regola generica è:

! Regole per scrivere una formula in R

- L'uguale si rimpiazza con il simbolo `~`.
- Gli oggetti a sinistra del simbolo `~` rappresentano la **variabile dipendente** (in questo caso y).

- Gli oggetti a destra del simbolo `~` rappresentano le **variabili indipendenti**, o *predittori* (in questo caso x, x^2, x^3, ecc.), separati da `+` ed eliminando i coefficienti (in questo caso a, b, c) che saranno stimati dal modello.
- L'**intercetta** (il termine a) è implicita e non va scritta nella formula. Se si vuole un modello **senza** intercetta, bisogna aggiungere `- 1` alla formula.
- I residui non vanno scritti nella formula, ma sono impliciti nel modello.

La formula va poi passata come primo argomento alla funzione `lm()` (*linear model*) che calcola i parametri del modello. Il secondo argomento della funzione è il dataset che contiene le variabili:

```
lm(y ~ x + x^2, data=df)
```

```
Call:
lm(formula = y ~ x + x^2, data = df)

Coefficients:
(Intercept)            x
      1.908        3.008
```

Osserviamo che la funzione `lm()` ha calcolato solo i coefficienti per l'intercetta e per x, ma non per x^2. Questo perché all'interno di una formula gli operatori `^`, `*` e `:` **non** hanno il loro significato aritmetico, ma servono a indicare le *interazioni* tra più variabili (come vedremo più avanti con i modelli a più variabili). In particolare, `(x + z)^2` in una formula significa "tutti i termini `x` e `z` e la loro interazione `x:z`", e quindi `x^2` per una singola variabile si riduce semplicemente a `x`.

Per **proteggere** dall'espansione di x^2 come formula, dobbiamo racchiudere l'espressione tra parentesi e precederla con la funzione `I()` (identity). Proviamo un modello di grado maggiore del necessario, fino a x^4:

```
df.lm <- lm(y~x + I(x^2) + I(x^3) + I(x^4), data=df)
coef(df.lm)
```

```
  (Intercept)             x        I(x^2)        I(x^3)        I(x^4)
 0.8326865737  2.0211928898  0.1186638267 -0.0048121358  0.0002640569
```

```
summary(df.lm)
```

```
Call:
lm(formula = y ~ x + I(x^2) + I(x^3) + I(x^4), data = df)
```

```
Residuals:
   Min     1Q Median     3Q    Max
-4.344 -1.206 -0.056  1.162  4.928

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept)  0.8326866  0.3478351   2.394   0.0186 *
x            2.0211929  0.1070973  18.872  < 2e-16 ***
I(x^2)       0.1186638  0.0227324   5.220 1.05e-06 ***
I(x^3)      -0.0048121  0.0045966  -1.047   0.2978
I(x^4)       0.0002641  0.0002227   1.185   0.2388
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.764 on 95 degrees of freedom
Multiple R-squared:  0.9908,    Adjusted R-squared:  0.9904
F-statistic:  2560 on 4 and 95 DF,  p-value: < 2.2e-16
```

La funzione `coef()` restituisce i parametri stimati, mentre `summary()` riporta per ogni parametro anche l'errore standard e il p-value di un test di Student la cui ipotesi nulla è che il parametro sia **nullo**. I termini `I(x^3)` e `I(x^4)` hanno p-value elevati: non sono significativi e vanno rimossi dal modello, che si riduce quindi al secondo grado. In fondo al sommario troviamo anche il coefficiente di determinazione R^2, che indica la frazione della varianza di y spiegata dal modello.

Una forma più breve per scrivere il modello polinomiale è usare la funzione `poly()`, che genera tutti i termini fino al grado indicato:

```r
df.lm <- lm(y~poly(x, 2), data=df)
coef(df.lm)
```

```
(Intercept) poly(x, 2)1 poly(x, 2)2
   16.94601   175.39224    33.20177
```

```r
summary(df.lm)
```

```
Call:
lm(formula = y ~ poly(x, 2), data = df)

Residuals:
    Min      1Q  Median      3Q     Max
-4.2214 -1.1822 -0.1493  1.2671  5.1431

Coefficients:
```

```
            Estimate Std. Error t value Pr(>|t|)
(Intercept)   16.946      0.176   96.27   <2e-16 ***
poly(x, 2)1  175.392      1.760   99.64   <2e-16 ***
poly(x, 2)2   33.202      1.760   18.86   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.76 on 97 degrees of freedom
Multiple R-squared:  0.9907,    Adjusted R-squared:  0.9905
F-statistic:  5142 on 2 and 97 DF,  p-value: < 2.2e-16
```

I coefficienti però non corrispondono ai valori nominali $a = 1$, $b = 2$, $c = 0.1$! Questo perché per default `poly()` usa dei polinomi **ortogonali**, e non i polinomi **grezzi** (*raw*) $x, x^2, ...$:

- i polinomi **grezzi** si ottengono con `poly(x, 2, raw=TRUE)`, che è equivalente a `x + I(x^2)`. I coefficienti stimati corrispondono direttamente ai parametri a, b, c del modello, ma i termini $x, x^2, x^3, ...$ sono fortemente **correlati** tra loro: aggiungendo o togliendo un termine cambiano in generale le stime degli altri coefficienti, e per gradi elevati il calcolo diventa numericamente instabile;
- i polinomi **ortogonali** sono combinazioni lineari di $x, x^2, ...$ costruite in modo da essere tra loro non correlati sui valori di x osservati. I coefficienti non hanno un'interpretazione fisica diretta, ma sono indipendenti tra loro: aggiungendo un grado al modello, le stime dei coefficienti di grado inferiore **non cambiano**, e il calcolo è numericamente stabile.

Verifichiamolo confrontando i coefficienti dei modelli di grado 2 e 3, nelle due versioni:

```r
# polinomi ortogonali: i primi tre coefficienti restano invariati
cat("Polinomi ortogonali:")
```

```
Polinomi ortogonali:
```

```r
lm(y~poly(x, 2), data=df) %>% coef()
```

```
(Intercept) poly(x, 2)1 poly(x, 2)2
   16.94601    175.39224     33.20177
```

```r
lm(y~poly(x, 3), data=df) %>% coef()
```

```
(Intercept) poly(x, 3)1 poly(x, 3)2 poly(x, 3)3
 16.9460103 175.3922404  33.2017746   0.7302588
```

```r
# polinomi grezzi: i coefficienti cambiano
cat("\nPolinomi grezzi:")
```

```
Polinomi grezzi:
```

```r
lm(y~poly(x, 2, raw=TRUE), data=df) %>% coef()
```

```
            (Intercept) poly(x, 2, raw = TRUE)1
              0.9244438               1.9159210
poly(x, 2, raw = TRUE)2
              0.1091733
```

```r
lm(y~poly(x, 3, raw=TRUE), data=df) %>% coef()
```

```
            (Intercept) poly(x, 3, raw = TRUE)1
           1.0093426310            1.9223914047
poly(x, 3, raw = TRUE)2 poly(x, 3, raw = TRUE)3
           0.1021382819            0.0004690029
```

Con i polinomi grezzi, aggiungendo il termine di terzo grado cambia la stima del coefficiente di x. In questo caso particolare intercetta e coefficiente di x^2 restano invariati solo perché i valori di x sono simmetrici rispetto allo zero, e quindi i termini di grado pari sono non correlati con quelli di grado dispari; con valori di x qualsiasi (ad esempio tra 0 e 20) cambierebbero tutti i coefficienti.

In ogni caso, i due modelli dello stesso grado sono **equivalenti**: forniscono esattamente le stesse previsioni e gli stessi residui, e differiscono solo per il modo in cui sono espressi i coefficienti. Possiamo verificarlo confrontando i valori previsti dai due modelli di grado 2 usando:

- `fitted()`, che restituisce i valori previsti dal modello per ciascun valore di x osservato;
- `all.equal()`, che confronta due oggetti e restituisce `TRUE` se sono uguali (entro una tolleranza numerica):

```r
all.equal(
  fitted(lm(y~poly(x, 2), data=df)),
  fitted(lm(y~poly(x, 2, raw=TRUE), data=df))
)
```

```
[1] TRUE
```

I polinomi ortogonali sono anche più robusti nell'analisi di **significatività** dei parametri, cioè quando si usano i p-value del sommario per decidere quali termini mantenere nel modello. Il motivo è la **correlazione** tra i termini: verifichiamo con `cor()` la matrice di correlazione dei primi quattro termini nelle due versioni:

```
# polinomi grezzi: x e x^3, così come x^2 e x^4, sono fortemente correlati
cor(poly(df$x, 4, raw=TRUE)) %>% round(3)
```

```
      1     2     3     4
1 1.000 0.887 0.855 0.786
2 0.887 1.000 0.985 0.958
3 0.855 0.985 1.000 0.990
4 0.786 0.958 0.990 1.000
```

```
# polinomi ortogonali: la matrice di correlazione è la matrice identità
cor(poly(df$x, 4)) %>% round(3)
```

```
  1 2 3 4
1 1 0 0 0
2 0 1 0 0
3 0 0 1 0
4 0 0 0 1
```

Quando due predittori sono fortemente correlati (si parla di **multicollinearità**), il modello fatica a distinguere il contributo dell'uno da quello dell'altro: le stime dei relativi coefficienti diventano molto incerte, cioè i loro errori standard crescono, e di conseguenza crescono anche i p-value. Un termine realmente significativo può quindi *sembrare* non significativo solo perché nel modello ci sono altri termini correlati con esso.

Confrontiamo i sommari dei modelli di grado 2 (quello corretto) e di grado 7 (sovradimensionato), nelle due versioni:

```
# polinomi ortogonali: stime ed errori standard dei primi termini
# restano praticamente invariati
cat("Polinomi ortogonali:")
```

```
Polinomi ortogonali:
```

```
lm(y~poly(x, 2), data=df) %>% summary()
```

```
Call:
lm(formula = y ~ poly(x, 2), data = df)

Residuals:
    Min      1Q  Median      3Q     Max
-4.2214 -1.1822 -0.1493  1.2671  5.1431

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)   16.946      0.176   96.27   <2e-16 ***
poly(x, 2)1  175.392      1.760   99.64   <2e-16 ***
poly(x, 2)2   33.202      1.760   18.86   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.76 on 97 degrees of freedom
Multiple R-squared:  0.9907,    Adjusted R-squared:  0.9905
F-statistic:  5142 on 2 and 97 DF,  p-value: < 2.2e-16
```

```r
lm(y~poly(x, 7), data=df) %>% summary()
```

```
Call:
lm(formula = y ~ poly(x, 7), data = df)

Residuals:
    Min      1Q  Median      3Q     Max
-3.9553 -1.2232 -0.0623  1.1339  5.1798

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  16.9460     0.1768  95.825   <2e-16 ***
poly(x, 7)1 175.3922     1.7684  99.179   <2e-16 ***
poly(x, 7)2  33.2018     1.7684  18.775   <2e-16 ***
poly(x, 7)3   0.7303     1.7684   0.413    0.681
poly(x, 7)4   2.0912     1.7684   1.183    0.240
poly(x, 7)5  -2.2585     1.7684  -1.277    0.205
poly(x, 7)6   1.2895     1.7684   0.729    0.468
poly(x, 7)7   1.0684     1.7684   0.604    0.547
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.768 on 92 degrees of freedom
Multiple R-squared:  0.9911,    Adjusted R-squared:  0.9904
F-statistic:  1456 on 7 and 92 DF,  p-value: < 2.2e-16
```

```r
# polinomi grezzi: aggiungendo termini, le stime cambiano e gli errori
# standard dei primi termini crescono
cat("\nPolinomi grezzi:")
```

```
Polinomi grezzi:
```

```r
lm(y~poly(x, 2, raw=TRUE), data=df) %>% summary()
```

```
Call:
lm(formula = y ~ poly(x, 2, raw = TRUE), data = df)

Residuals:
    Min      1Q  Median      3Q     Max
-4.2214 -1.1822 -0.1493  1.2671  5.1431

Coefficients:
                      Estimate Std. Error t value Pr(>|t|)
(Intercept)           0.924444   0.237648    3.89 0.000184 ***
poly(x, 2, raw = TRUE)1 1.915921   0.065276   29.35  < 2e-16 ***
poly(x, 2, raw = TRUE)2 0.109173   0.005788   18.86  < 2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.76 on 97 degrees of freedom
Multiple R-squared:  0.9907,    Adjusted R-squared:  0.9905
F-statistic:  5142 on 2 and 97 DF,  p-value: < 2.2e-16
```

```r
lm(y~poly(x, 7, raw=TRUE), data=df) %>% summary()
```

```
Call:
lm(formula = y ~ poly(x, 7, raw = TRUE), data = df)

Residuals:
    Min      1Q  Median      3Q     Max
-3.9553 -1.2232 -0.0623  1.1339  5.1798

Coefficients:
                        Estimate Std. Error t value Pr(>|t|)
(Intercept)            9.709e-01  4.357e-01   2.228   0.0283 *
poly(x, 7, raw = TRUE)1  2.290e+00  2.190e-01  10.459   <2e-16 ***
poly(x, 7, raw = TRUE)2  7.738e-02  7.964e-02   0.972   0.3338
poly(x, 7, raw = TRUE)3 -2.286e-02  2.160e-02  -1.058   0.2928
```

```
poly(x, 7, raw = TRUE)4  2.897e-03  2.385e-03   1.215   0.2274
poly(x, 7, raw = TRUE)5  1.286e-04  6.754e-04   0.190   0.8494
poly(x, 7, raw = TRUE)6 -3.010e-05  6.096e-05  -0.494   0.6226
poly(x, 7, raw = TRUE)7  1.041e-06  1.724e-06   0.604   0.5472
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.768 on 92 degrees of freedom
Multiple R-squared:  0.9911,    Adjusted R-squared:  0.9904
F-statistic:  1456 on 7 and 92 DF,  p-value: < 2.2e-16
```

Osserviamo che:

- con i polinomi **ortogonali**, i termini di primo e secondo grado hanno la stessa stima e praticamente lo stesso errore standard in entrambi i modelli, e restano nettamente significativi; tutti i termini dal terzo grado in su hanno invece p-value elevati. Il sommario del modello di grado 7 indica quindi in modo univoco che il modello corretto è di secondo grado;
- con i polinomi **grezzi**, nel modello di grado 7 l'errore standard del coefficiente di x^2 è circa 8 volte più grande che nel modello di grado 2, e il suo p-value passa da un valore praticamente nullo a circa 0.01. Con un livello di significatività più severo (ad esempio 1%) concluderemmo erroneamente che il termine quadratico non è significativo, e se rimuovessimo i termini uno alla volta il risultato dipenderebbe dall'ordine in cui lo facciamo;
- il p-value del termine di grado **massimo** è invece lo stesso nelle due versioni: è l'unico termine che le due basi polinomiali rappresentano allo stesso modo, cioè come il contributo aggiuntivo rispetto a tutti i termini di grado inferiore.

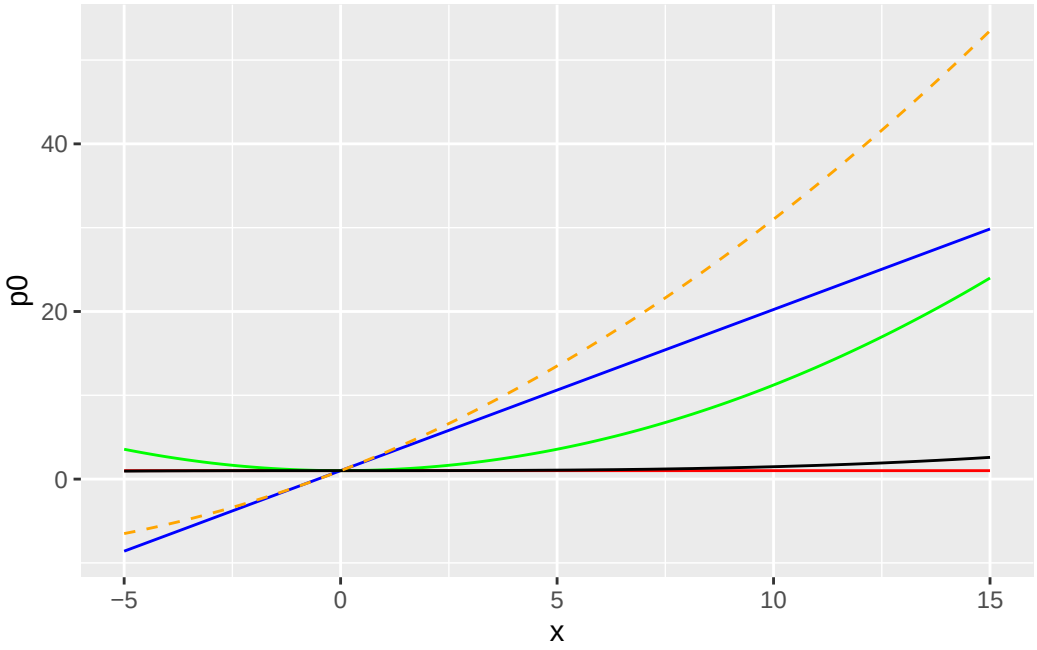
Per questo motivo, per decidere quale grado del polinomio sia necessario conviene stimare un modello sovradimensionato con polinomi ortogonali, individuare il grado massimo significativo e infine, se servono i valori fisici dei coefficienti, stimare il modello ridotto con polinomi grezzi.

> **Consiglio**
>
> In pratica, si usano i polinomi grezzi quando interessano i valori dei coefficienti, e quelli ortogonali quando interessa solo il modello (ad esempio per fare previsioni) o per gradi elevati, e per analizzare la significatività (p-value) dei vari contributi.

Graficamente, per un polinomio grezzo:

```r
df.lm <- lm(y~poly(x, 3, raw=T), data=df)
df.c <- coef(df.lm)
df %>% mutate(
  p0 = df.c[1],
  p1 = df.c[2] * x,
```

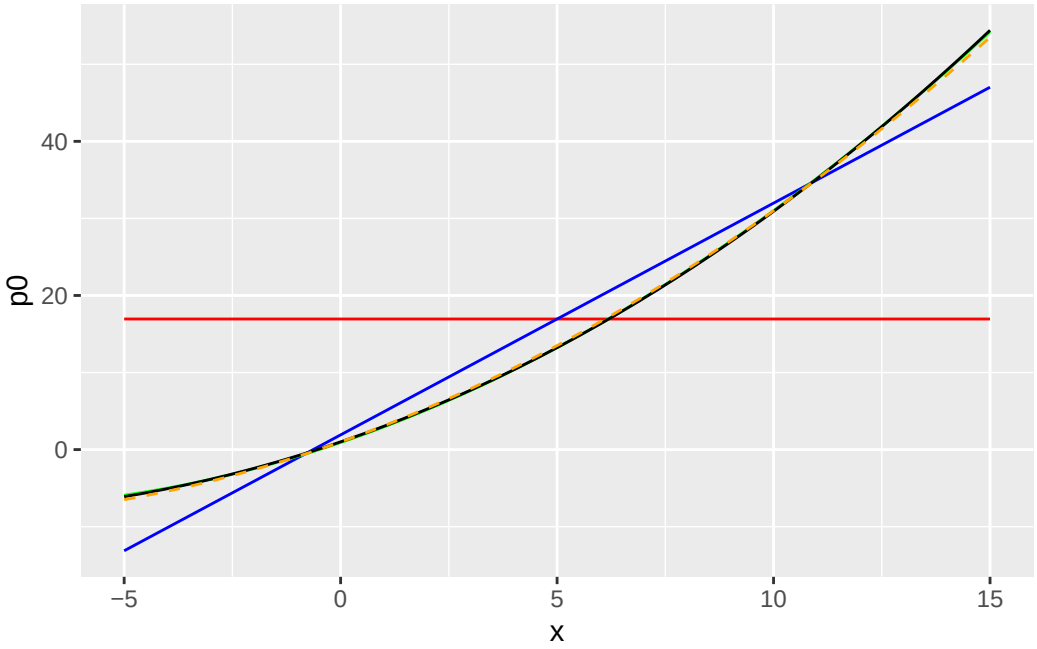
```
  p2 = df.c[3] * x^2,
  p3 = df.c[4] * x^3
) %>%
  ggplot(aes(x=x)) +
  geom_line(aes(y=p0), color="red") +
  geom_line(aes(y=p0+p1), color="blue") +
  geom_line(aes(y=p0+p2), color="green") +
  geom_line(aes(y=p0+p3), color="black") +
  geom_line(aes(y=y_nom), color="orange", lty=2)
```



Invece, per un polinomio ortogonale ogni contributo è $\beta_k P_k(x)$, dove P_k è il polinomio ortogonale di grado k e β_k il coefficiente stimato. La somma dei contributi fino al grado k fornisce la stima del modello di grado k:

```
df.lmo <- lm(y~poly(x, 3), data=df)
df.co <- coef(df.lmo)
P <- poly(df$x, 3)   # same basis used inside lm (same x values)
df %>% mutate(
  p0 = df.co[1],
  p1 = df.co[2] * P[,1],
  p2 = df.co[3] * P[,2],
  p3 = df.co[4] * P[,3]
) %>%
  ggplot(aes(x=x)) +
  geom_line(aes(y=p0), color="red") +
```

```
  geom_line(aes(y=p0+p1), color="blue") +
  geom_line(aes(y=p0+p1+p2), color="green") +
  geom_line(aes(y=p0+p1+p2+p3), color="black") +
  geom_line(aes(y=y_nom), color="orange", lty=2)
```



Si noti che ogni componente rappresenta **esattamente** la regressione dei dati fino al grado corrispondente.

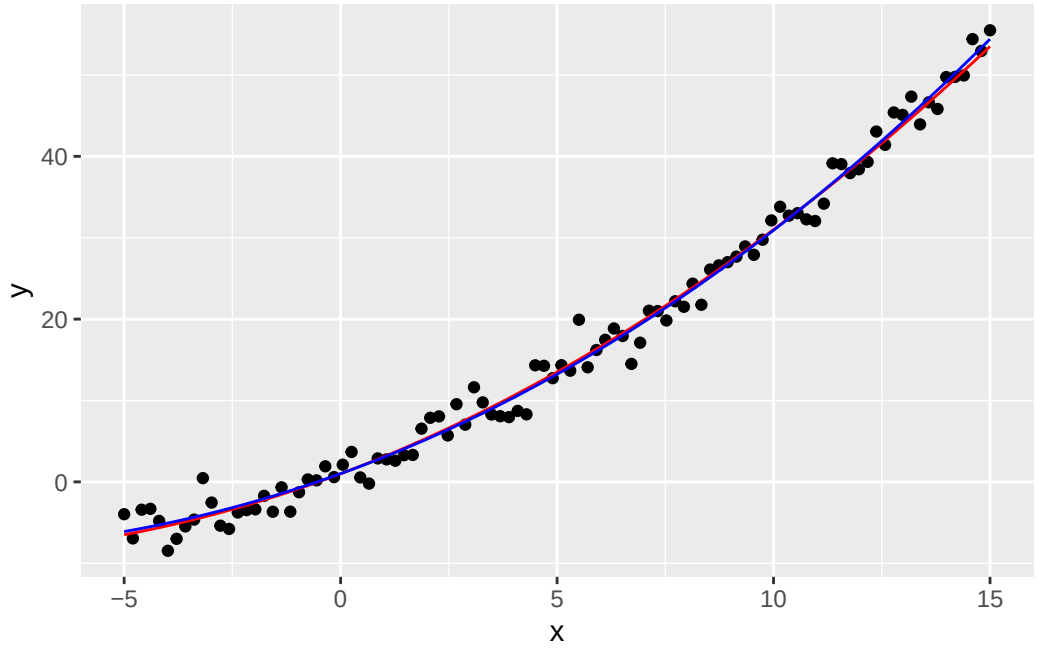
> **Esercizio**
>
> Ripetere gli ultimidue grafici trasformando la tabella in forma *tidy* e usando un'unica geometria `geom_line()` con l'estetica `color` mappata sulla colonna che indica il grado del polinomio. Aggiungere anche i punti dei dati osservati.

Il prossimo passo è utilizzare il modello per predire i valori di y e verificare la normalità dei residui. Le funzioni `add_predictions()` e `add_residuals()` (della libreria `adas.utils`) aggiungono a una tabella rispettivamente la colonna `pred` con i valori previsti dal modello e la colonna `resid` con i residui, cioè le differenze tra i valori osservati e quelli previsti:

```
df2 <- df %>%
  add_predictions(df.lm) %>%
  add_residuals(df.lm)

# in rosso la relazione nominale, in blu il modello stimato
df2 %>% ggplot(aes(x=x)) +
```

```
  geom_point(aes(y=y)) +
  geom_line(aes(y=y_nom), color="red") +
  geom_line(aes(y=pred), color="blue")
```



Il modello è valido solo se i residui rispettano le ipotesi fatte, cioè se sono distribuiti normalmente (e, come vedremo, se non mostrano andamenti sistematici). Verifichiamo la normalità con il test di Shapiro-Wilk:

```
shapiro.test(df2$resid)
```

```
    Shapiro-Wilk normality test

data:  df2$resid
W = 0.98951, p-value = 0.6248
```

> **Esercizio**
>
> Verificare la normalità dei residui con metodi grafici come il diagramma Q-Q e l'istogramma.

1.2 Estrapolazione

Un modello di regressione è valido solo all'interno dell'intervallo dei dati usati per stimarlo. Usarlo al di fuori di questo intervallo si chiama **estrapolazione**, ed è un'operazione

rischiosa. Per mostrarlo, dividiamo i dati in due sottoinsiemi: quelli interni all'intervallo $(-7.5, 7.5)$ (`in`) e quelli esterni (`out`):

```
df3 <- df %>%
  mutate(
    subset = ifelse(x > -2.5 & x < 13.5, "in", "out")
  )

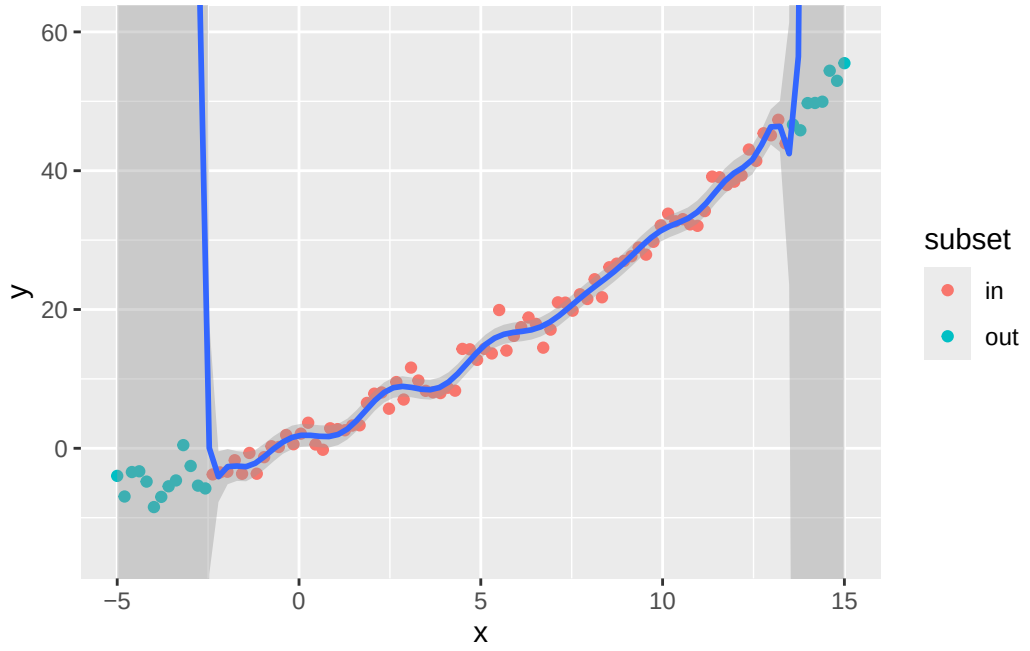
df3 %>%
  ggplot(aes(x=x, y=y, color=subset)) +
  geom_point()
```



Ora stimiamo un modello usando **solo** i dati interni, e lo rappresentiamo su tutto l'intervallo. La funzione `geom_smooth()` stima e disegna direttamente un modello di regressione (con `method="lm"`) secondo la formula indicata; l'opzione `fullrange=TRUE` estende la curva a tutto l'intervallo dell'asse x. Volutamente scegliamo un polinomio di grado molto elevato (21):

```
df3 %>%
  ggplot(aes(x=x, y=y)) +
  geom_point(aes(color=subset)) +
  geom_smooth(
    data=filter(df3, subset == "in"),
    method="lm",
    formula= y~poly(x, 21),
    fullrange=TRUE
```

```
  ) +
  coord_cartesian(ylim=c(-15,60))
```



All'interno dell'intervallo il modello segue (anche troppo) fedelmente i dati, inseguendo il rumore casuale: si parla di **sovradattamento** (*overfitting*). Appena fuori dall'intervallo, invece, la curva diverge rapidamente e le previsioni non hanno più alcun senso. La funzione `coord_cartesian()` limita l'area visualizzata senza eliminare dati, altrimenti la curva renderebbe illeggibile il grafico.

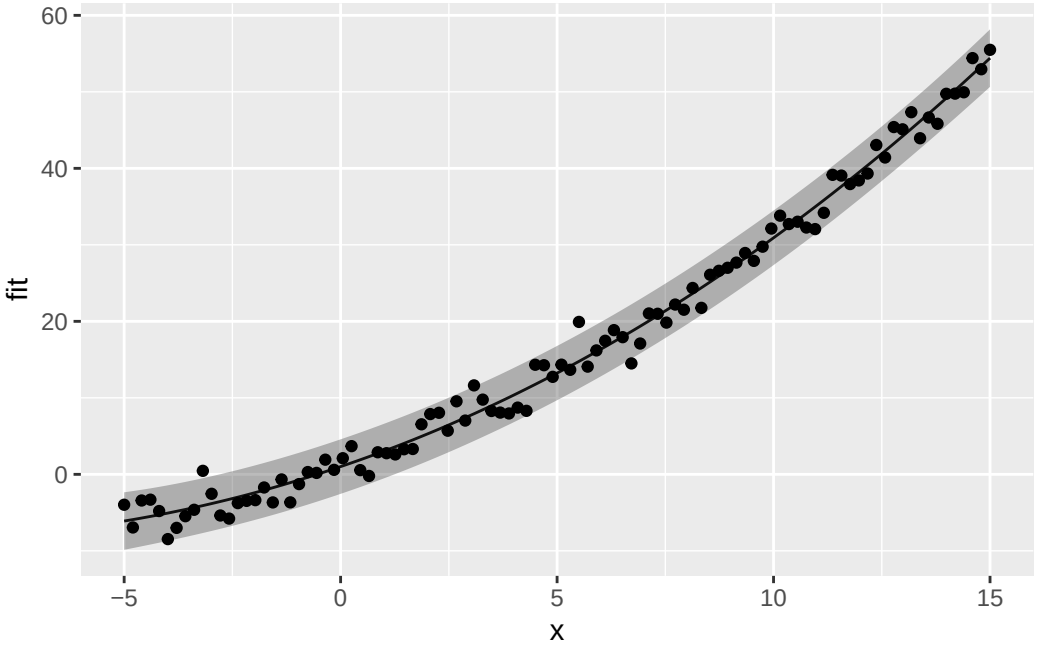
> **Esercizio**
>
> Ripetere il grafico con polinomi di grado 1, 2 e 5 e commentare il risultato.

La geometria `geom_smooth()` è comoda per fare una stima rapida, ma fornisce soltanto un risultato grafico. Per fare previsioni e analizzare i residui, bisogna stimare il modello con `lm()` e poi usare `add_predictions()` e `add_residuals()` come visto sopra.

Con l'opzione `interval="prediction"`, la funzione `add_predictions()` aggiunge le colonne `fit` (valore previsto), `lwr` e `upr` (limiti inferiore e superiore dell'**intervallo di predizione**, al 95% per default). L'intervallo di predizione è la banda entro cui ci aspettiamo che cada una **nuova osservazione** di y, per un dato x:

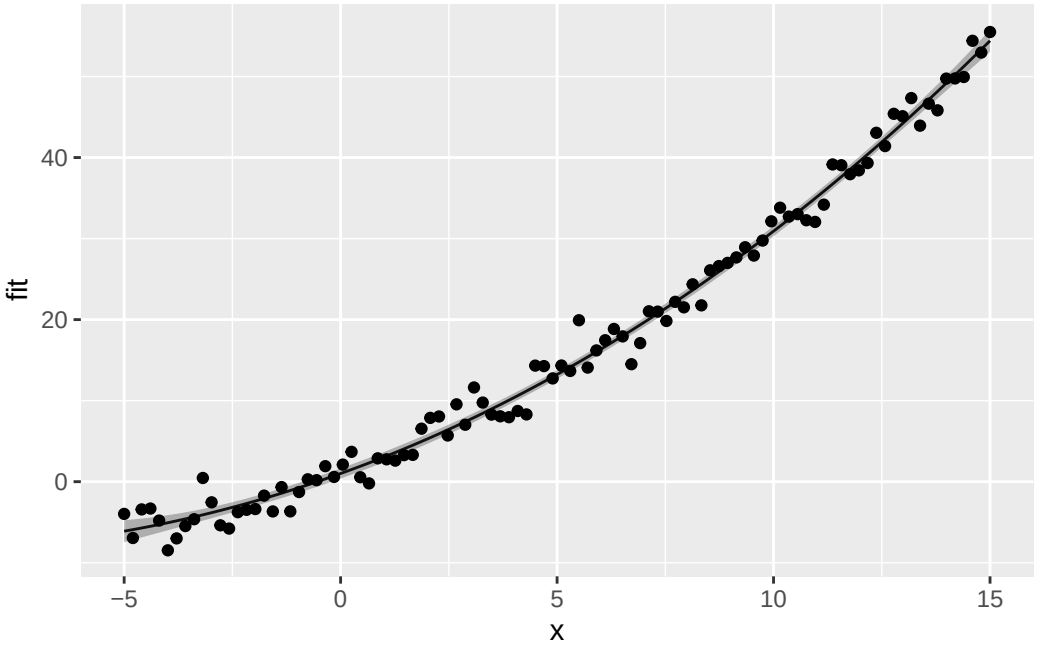
```
df %>%
  add_predictions(df.lm, interval="prediction") %>%
  ggplot(aes(x=x)) +
  geom_line(aes(y=fit)) +
```

```
  geom_ribbon(aes(ymin=lwr, ymax=upr), alpha=1/3) +
  geom_point(aes(y=y))
```



Con `interval="confidence"` si ottiene invece l'**intervallo di confidenza**, cioè la banda entro cui si trova, con il 95% di probabilità, il **valore atteso** di y (la curva vera). L'intervallo di confidenza è sempre più stretto di quello di predizione, perché quest'ultimo tiene conto anche della dispersione σ delle singole osservazioni:

```
df %>%
  add_predictions(df.lm, interval="confidence") %>%
  ggplot(aes(x=x)) +
  geom_line(aes(y=fit)) +
  geom_ribbon(aes(ymin=lwr, ymax=upr), alpha=1/3) +
  geom_point(aes(y=y))
```



1.3 Regressione ai minimi quadrati

Non tutti i modelli possono essere espressi come modelli lineari nei parametri. Consideriamo ad esempio un **problema di contatto**: misuriamo la forza tra due corpi (es. sfera e piano) inizialmente separati mentre si avvicinano, e registriamo la forza f in funzione della posizione y. Si tratta di una funzione che vale costantemente `bias` fino a y_0, e da y_0 in poi prosegue come una parabola con vertice in (y_0, bias) e coefficiente quadratico a:

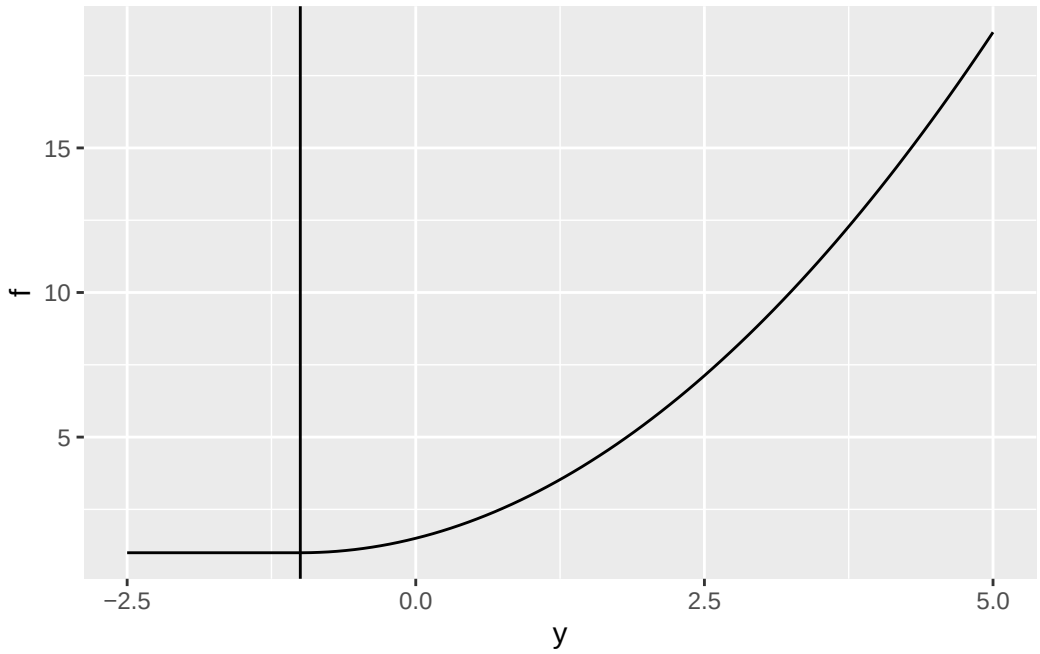
$$f(y) = \begin{cases} \text{bias} & y < y_0 \\ \text{bias} + a(y - y_0)^2 & y \geq y_0 \end{cases}$$

Il parametro y_0 compare in modo non lineare (e anche all'interno di una condizione), quindi `lm()` non è utilizzabile. Definiamo la funzione in R, sviluppando il quadrato nei coefficienti b e c della parabola, e rappresentiamola:

```
f <- function(y, y0=0, bias=0, a=1) {
  b <- -2 *a *y0
  c <- bias + a * y0^2
  f <- a * y^2 + b * y + c
  # ifelse() è la versione vettorializzata di if-else
  return(ifelse(y<y0, bias, f))
}

tibble(
```

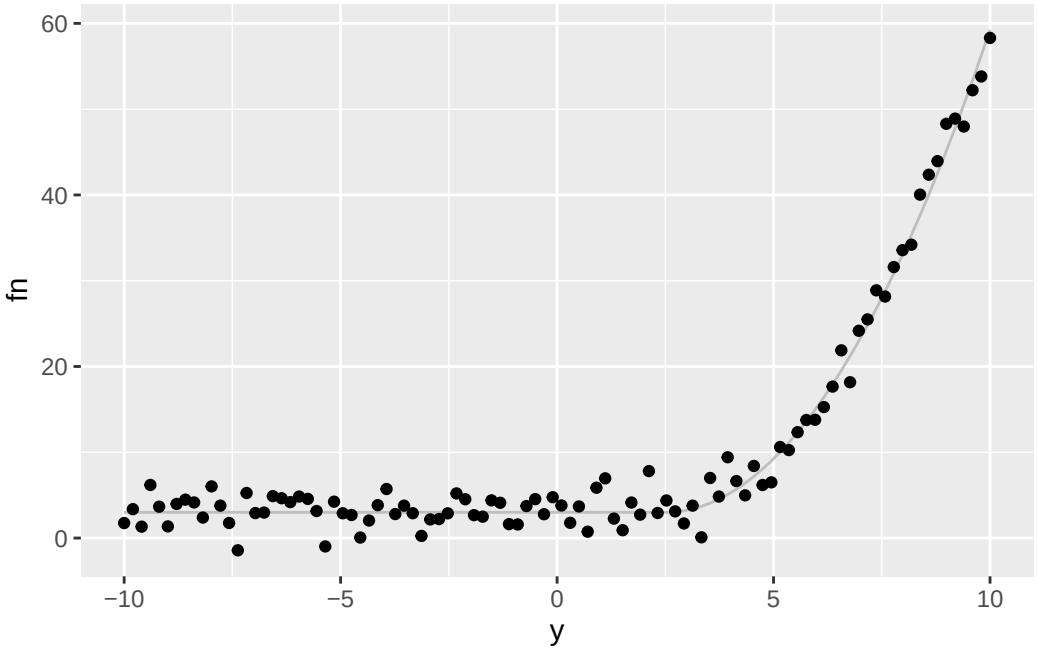
```
  y = seq(-2.5, 5, length.out=100),
  f = f(y, -1, 1, 0.5)
) %>%
  ggplot(aes(x=y, y=f)) +
  geom_line() +
  geom_vline(xintercept=-1)
```



Generiamo ora dei dati sintetici con $y_0 = 2.5$, bias $= 3$ e $a = 1$, aggiungendo un disturbo normale:

```
set.seed(1)
df <- tibble(
  y = seq(-10, 10, length.out=100),
  fn = f(y, 2.5, 3, 1),
  f = fn + rnorm(length(y), 0, 2)
)

df %>% ggplot(aes(x=y)) +
  geom_line(aes(y=fn), color=grey(3/4)) +
  geom_point(aes(y=f))
```



Si noti che dai soli dati è molto difficile individuare il punto di contatto y_0, perché la parabola è molto piatta vicino al minimo. Tale valore può però essere **identificato** mediante una regressione non lineare, che stima i parametri y_0, bias e a a partire dai dati osservati.

Il metodo dei **minimi quadrati** consiste nel cercare i valori dei parametri che rendono minima la somma dei quadrati dei residui:

$$\Phi(y_0, \text{bias}, a) = \sum_{i=1}^{N} (f_i - f(y_i; y_0, \text{bias}, a))^2$$

Per i modelli lineari il minimo si ottiene analiticamente (ed è proprio ciò che fa `lm()`); nel caso generale bisogna invece cercarlo **numericamente**. Definiamo la funzione obiettivo Z, che prende come primo argomento il vettore dei parametri, e la minimizziamo con la funzione `optim()`, partendo da una stima iniziale dei parametri:

```
Phi <- function(par, data) {
  sum((data$f - f(data$y, par["y0"], par["bias"], par["a"]))^2)
}

fit <- optim(c(y0=0, bias=0, a=1), Phi, data=df)
fit$par         # parametri stimati
```

```
      y0     bias        a
2.505737 3.258401 0.996786
```

```r
fit$convergence # 0 indica che l'ottimizzazione è andata a buon fine
```

```
[1] 0
```

R mette anche a disposizione la funzione `nls()` (*non-linear least squares*), che usa una formula come `lm()`. A destra della `~` si scrive l'espressione del modello, indicando i parametri per nome, e con l'argomento `start` si forniscono i valori iniziali dei parametri da stimare. Rispetto a `optim()`, `nls()` fornisce anche gli errori standard e i p-value dei parametri:

```r
# NB: a sinistra della ~ c'è la colonna f, a destra la funzione f()
df.nls <- nls(f ~ f(y, y0, bias, a), data=df, start=list(y0=0, bias=0,
 ↪  a=1))
summary(df.nls)
```

```
Formula: f ~ f(y, y0, bias, a)

Parameters:
     Estimate Std. Error t value Pr(>|t|)
y0    2.50571    0.14758   16.98   <2e-16 ***
bias  3.25779    0.22222   14.66   <2e-16 ***
a     0.99672    0.04765   20.92   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.813 on 97 degrees of freedom

Number of iterations to convergence: 5
Achieved convergence tolerance: 1.91e-06
```

! Dov'è R^2?

Per la regressione non lineare non esiste un equivalente del coefficiente di determinazione R^2 dei modelli lineari. In altre parole, per una regressione non lineare si possono avere valori di R^2 negativi, oppure regressioni palesemente errate con valori di R^2 molto vicini a 1. Per questo motivo, per valutare la bontà del modello conviene sempre rappresentare graficamente i dati osservati e quelli previsti dal modello, e verificare la normalità dei residui.
Per confrontare regressioni ottenute con differenti modelli non lineari è possibile usare il criterio di informazione di Akaike (AIC), che tiene conto sia della bontà del modello che del numero di parametri stimati. **Più basso è il valore di AIC, migliore è il modello**. In R si calcola con la funzione `AIC()`:

```
AIC(df.nls)
```

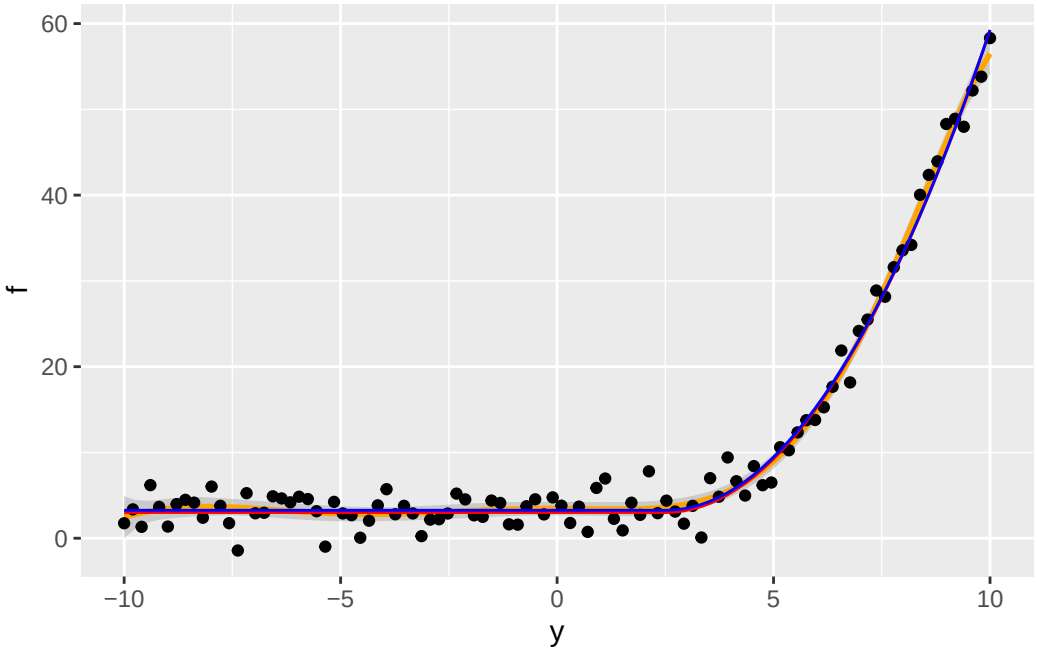
```
[1] 407.7792
```

```
AIC(lm(f~poly(y, 7), data=df))
```

```
[1] 409.8989
```

Il modello stimato si usa come quelli lineari, ad esempio per aggiungere previsioni e residui alla tabella:

```
df %>%
  add_predictions(df.nls) %>%
  ggplot(aes(x=y)) +
  geom_smooth(
    aes(y=f), color="orange", method="lm", formula=y~poly(x, 7)) +
  geom_point(aes(y=f)) +
  geom_line(aes(y=fn), color="red") +
  geom_line(aes(y=pred), color="blue")
```



Si noti che abbiamo aggiunto anche la stima di un modello lineare di grado 7, per mostrare che il modello non lineare stimato con `nls()` segue meglio i dati osservati, giustificando i valori di AIC più bassi sopra riportati.

> **Avviso**
>
> Gli algoritmi di ottimizzazione numerica sono **iterativi** e partono dai valori iniziali forniti: se questi sono troppo lontani dalla soluzione, l'algoritmo può non convergere o convergere a un minimo locale. È quindi buona norma scegliere valori iniziali ragionevoli (ad esempio stimandoli dal grafico dei dati) e verificare sempre il risultato graficamente.

> **Esercizio**
>
> Verificare la normalità dei residui del modello `df.nls`, e provare a ripetere la stima con valori iniziali diversi (ad esempio `y0=-5`).

1.4 Regressione lineare generalizzata

Carichiamo la tabella di dati di esempio "soap_bottles.dat". Essa contiene i risultati di un *drop test* su bottiglie di sapone, in cui si verifica la sopravvivenza della bottiglia al variare del livello di riempimento p rispetto alla capacità massima dopo una caduta da 1 m di altezza. La colonna `OK` indica se la bottiglia è sopravvissuta (`TRUE`) o meno (`FALSE`).

Infatti, il polmone d'aria della bottiglia consente di assorbire le onde di sovrapressione generate dall'urto, e quindi la probabilità di sopravvivenza dipende dal livello di riempimento (oltre che dalle specificità aleatorie della singola bottiglia). La relazione tra la probabilità di fallimento e il livello di riempimento è tipicamente **sigmoide**, e può essere modellata con una **regressione logistica**, che tipicamente rappresenta dei processi, come questo, **bimodali**.

Per comodità, Creiamo una nuova colonna `fail` che indica il fallimento del test, e rimuoviamo la colonna `OK`:

```
data <- examples_url("soap_bottles.dat") %>%
  read.table(header=T, comment="#") %>%
  mutate(
    fail = !OK
  ) %>%
  select(-OK)

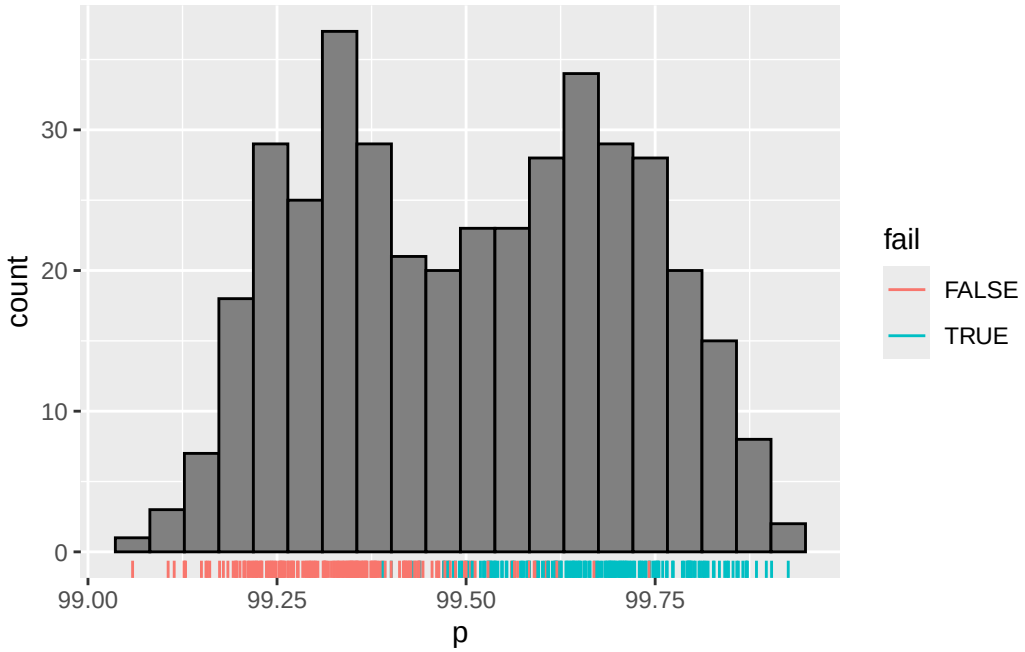
data %>% head()
```

```
  run      p  fail
1   1 99.314 FALSE
2   2 99.357 FALSE
3   3 99.624 FALSE
4   4 99.429  TRUE
```

```
5   5 99.860  TRUE
6   6 99.212 FALSE
```

Osserviamo la distribuzione dei valori di `p` in un istogramma, con un *rug plot* che indica i valori di `p` per le bottiglie che hanno fallito/superato il test:

```
data %>%
  ggplot(aes(x=p)) +
  geom_histogram(bins=20, fill=gray(1/2), color="black") +
  geom_rug(aes(color=fail))
```



Il set di dati è abbastanza sostanzioso:

```
dim(data)
```

```
[1] 400   3
```

Possiamo quindi dividerlo in due parti: un sottoinsieme di **addestramento** (*training set*) per stimare i parametri del modello, e un sottoinsieme di **validazione** (*validation set*) per verificare la bontà del modello stimato. La divisione può essere fatta in modo casuale, ad esempio con una probabilità dell'80% di finire nel sottoinsieme di addestramento e del 20% in quello di validazione.

Per farlo, generiamo una colonna `training` che contiene etichette casuali "train" o "valid" per ciascun record, con probabilità 80% e 20% rispettivamente. La funzione `sample()`

genera un campione casuale di etichette, e l'argomento `replace=TRUE` indica che valore può essere estratto più volte (si dice *campionamento con reinserimento*):

```
sample(c("train", "valid"), size=5, replace=TRUE)
```

```
[1] "valid" "valid" "train" "valid" "valid"
```

Quindi:

```
set.seed(1)
data <- data %>%
  mutate(
    training=sample(c("train", "valid"), size=n(), prob=c(80, 20),
    ↪  replace=TRUE)
  )
```

Per verificare la distribuzione dei dati nei due sottoinsiemi, possiamo contare il numero di record in ciascun sottoinsieme mediante `dplyr::summarise()`:

```
data %>%
  summarise(
    n=n(),
    .by=training
  )
```

```
  training   n
1    train 328
2    valid  72
```

Finalmente possiamo regredire il modello generalizzato, usando come **link function** la funzione logit. In R, la regressione logistica si ottiene con la funzione `glm()` (*generalized linear model*), indicando come argomento `family="binomial"`:

```
data.glm <- glm(fail~p, family="binomial", data=filter(data,
 ↪  training=="train"))

summary(data.glm)
```

```
Call:
glm(formula = fail ~ p, family = "binomial", data = filter(data,
    training == "train"))
```

```
Coefficients:
             Estimate Std. Error z value Pr(>|z|)
(Intercept) -2213.654    260.927  -8.484   <2e-16 ***
p              22.245      2.622   8.484   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 454.51  on 327  degrees of freedom
Residual deviance: 132.28  on 326  degrees of freedom
AIC: 136.28

Number of Fisher Scoring iterations: 7
```

```r
coef(data.glm)
```

```
(Intercept)           p
-2213.65369    22.24533
```

La funzione logit è definita come:

$$\text{logit}(x) = \frac{1}{1 + e^{-(px+m)}}$$

Che è equivalente alla:

$$\text{logit}(x) = \frac{1}{1 + e^{-p(x-x_0)}}$$

dove l'ascissa del punto di flesso x_0 è legata ai coefficienti del modello dalla relazione:

$$x_0 = -m/p$$

Quindi:

```r
p <-  coef(data.glm)[2]
x0 <- -coef(data.glm)[1] / p
x0
```

```
(Intercept)
   99.51093
```

```
p
```

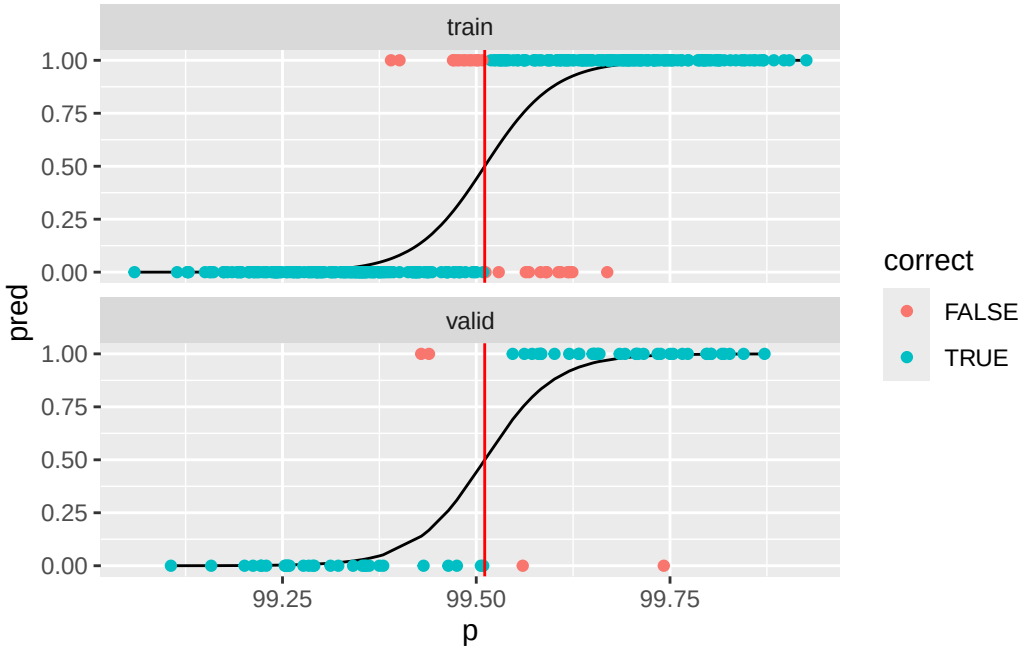
```
       p
22.24533
```

Per visualizzare la regressione aggiungiamo le predizioni alla tabella, specificando `type="response"` per ottenere le probabilità di fallimento (altrimenti si ottengono i valori della funzione logit senza la trasformazione inversa):

```
data <- data %>%
  add_predictions(data.glm, type="response")
```

Possiamo ora mettere in grafico i dati e la curva di regressione, distinguendo i due sottoinsiemi di addestramento e validazione. La linea rossa verticale indica il punto di flesso della curva e separa le due regioni di probabilità di fallimento maggiore o minore del 50%, con gli insiemi dei falsi positivi e dei falsi negativi equipopolati:

```
data %>%
  mutate(
    correct = (fail == (pred > 0.5))
  ) %>%
  ggplot(aes(x=p)) +
  geom_line(aes(y=pred)) +
  geom_point(aes(y=as.numeric(fail), color=correct)) +
  geom_vline(xintercept=x0, color="red") +
  facet_wrap(~training, nrow=2)
```



Si noti l'utilizzo di `facet_wrap()` per creare due grafici separati per i due sottoinsiemi, scegliendo i sottoinsiemi con `~training`, e di `as.numeric()` per convertire i valori booleani in numerici (0/1) in modo da poterli rappresentare sull'asse verticale.

Si osserva che gli errori di classificazione sono equiprobabili anche sull'insieme di validazione (due falsi positivi e due falsi negativi).

In maniera più quantitativa, possiamo contare e confrontare i vari casi:

```
error_type <- function(outcome, pred) {
  if (outcome == pred) return("correct")
  else if (outcome & !pred) return("false neg")
  else if (!outcome & pred) return("false pos")
}

data %>%
  mutate(
    type=map2_chr(fail, pred>0.5, error_type)
  ) %>%
  summarise(
    count=n(),
    .by = c(training, fail, type)
  ) %>%
  arrange(training, fail, type)
```

```
  training  fail      type count
1    train FALSE   correct   155
```

```
2    train FALSE false pos    13
3    train  TRUE   correct   146
4    train  TRUE false neg    14
5    valid FALSE   correct    30
6    valid FALSE false pos     2
7    valid  TRUE   correct    38
8    valid  TRUE false neg     2
```

Esercizio

Calcolare la tabella con le percentuali di falsi positivi e falsi negativi per i due sottoinsiemi.